

BASE DE DATOS



LABORATORIO 5

Funciones

Contenido

1. Funciones de manipulación de caracteres
2. Funciones de conversión de caracteres
3. Funciones numericas
4. Funciones de fecha
5. Formato de fechas
6. Operaciones con fechas
7. Cambion de tipos de datos
8. Función nvl
9. Función nvl2
10. Funcion nullif
11. Función coalesce
12. Función decode
13. Funciones de grupo
14. Manejo de valores nulos en las funciones de grupo
15. Agrupación de datos
16. Restringir los resultados de los grupos
17. Operador union
18. Operador union all
19. Operador intersect
20. Operador minus
21. Top – n
22. Rollup



FUNCIONES DE MANIPULACIÓN DE CARACTERES

Function	Result
CONCAT('Hello', 'World')	HelloWorld
SUBSTR('HelloWorld',1,5)	Hello
LENGTH('HelloWorld')	10
INSTR('HelloWorld', 'W')	6
LPAD(salary,10,'*')	*****24000
RPAD(salary, 10, '*')	24000*****
REPLACE ('JACK and JUE', 'J', 'BL')	BLACK and BLUE
TRIM('H' FROM 'HelloWorld')	elloWorld

1

```
SELECT employee_id, CONCAT(first_name, last_name) NAME,
       job_id, LENGTH(last_name),
       INSTR(last_name, 'a') "Contains 'a'?"
FROM   employees
WHERE  SUBSTR(job_id, 4) = 'REP';
```

2

3

EMPLOYEE_ID	NAME	JOB_ID	LENGTH(LAST_NAME)	Contains 'a'?
174	EllenAbel	SA_REP	4	0
176	JonathanTaylor	SA_REP	6	2
178	KimberelyGrant	SA_REP	5	3
202	PatFay	MK_REP	3	2

1

2

3

FUNCIONES DE CONVERSIÓN DE CARACTERES

Function	Result
LOWER('SQL Course')	sql course
UPPER('SQL Course')	SQL COURSE
INITCAP('SQL Course')	Sql Course

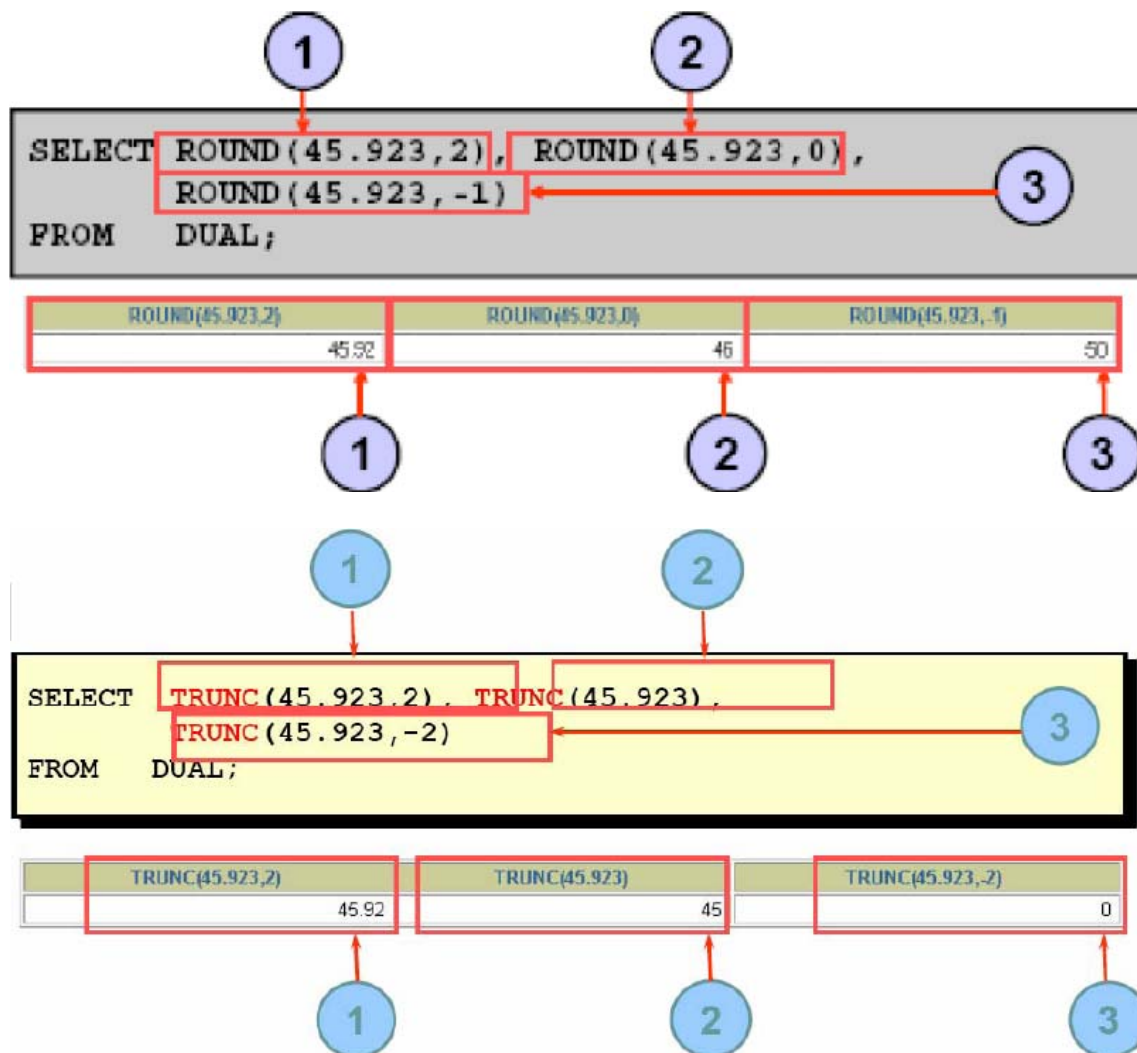
```
SELECT employee_id, last_name, department_id
FROM   employees
WHERE  LOWER(last_name) = 'higgins';
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
205	Higgins	110



FUNCIONES NUMERICAS

FUNCTION	RESULT
ROUND (45 . 926 , 2)	45 . 93
TRUNC (45 . 926 , 2)	45 . 92
MOD (1600 , 300)	100
POWER (2 , 4)	16
ABS (- 10)	10
SIGN (50)	1
SIGN (- 50)	- 1
SIGN (0)	0



DUAL es una tabla ficticia que se usa para ver los resultados de los cálculos.



FUNCIONES DE FECHA

Function	Result
MONTHS_BETWEEN	Number of months between two dates
ADD_MONTHS	Add calendar months to date
NEXT_DAY	Next day of the date specified
LAST_DAY	Last day of the month
ROUND	Round date
TRUNC	Truncate date

Function	Result
MONTHS_BETWEEN ('01-SEP-95', '11-JAN-94')	19.6774194
ADD_MONTHS ('11-JAN-94', 6)	'11-JUL-94'
NEXT_DAY ('01-SEP-95', 'FRIDAY')	'08-SEP-95'
LAST_DAY ('01-FEB-95')	'28-FEB-95'

Assume SYSDATE = '25-JUL-03':

Function	Result
ROUND (SYSDATE, 'MONTH')	01-AUG-03
ROUND (SYSDATE, 'YEAR')	01-JAN-04
TRUNC (SYSDATE, 'MONTH')	01-JUL-03
TRUNC (SYSDATE, 'YEAR')	01-JAN-03

FORMATO DE FECHAS

El formato de fecha usado por defecto es DD-MON-RR.

```
SELECT last_name, hire_date
FROM employees
WHERE hire_date < '01-FEB-88';
```

LAST_NAME	HIRE_DATE
King	17-JUN-87
Whalen	17-SEP-87

YYYY	año completo en números. Ej. 2004
YEAR	Año deletreado
MM	número de mes. Ej. 07
MONTH	nombre completo del mes. Ej. FEBRUARY
MON	abreviación de tres-letras del mes. Ej. FEB
DY	abreviación de tres-letras del día de la semana. Ej. WED
DAY	nombre completo del día de semana. Ej. MONDAY
DD	número de días en el mes. Ej. 16



Año Actual	Fech. especificada	FormatoRR	Formato YY
1995	27-OCT-95	1995	1995
1995	27-OCT-17	2017	1917
2010	27-OCT-17	2017	2017
2010	27-OCT-95	1995	2095

		Si el año especificado de dos-dígitos es:	
		0-49	50-99
Si dos dígitos del año actual son:	0-49	La fecha retornante esta en el siglo actual	La fecha retornante esta en el siglo antes del actual
	50-99	La fecha retornante esta en el siglo sgte. al siglo actual	La fecha retornante esta en el siglo actual

```
SELECT last_name, TO_CHAR(hire_date, 'DD-Mon-YYYY')
FROM employees
WHERE hire_date < TO_DATE('01-Jan-90', 'DD-Mon-RR');
```

SYSDATE es una funcion de fecha que no toma ningun input y retorna la fecha actual del sistema.

```
SELECT last_name, (SYSDATE-hire_date)/7 AS WEEKS
FROM employees
WHERE department_id = 90;
```

LAST_NAME	WEEKS
King	744 245395
Kochhar	626 102538
De Haan	453 245395

OPERACIONES CON FECHAS

```
SQL> select sysdate as Hoy, extract(year from sysdate) as Año
2 from dual;

HOY                                AÑO
-----
26-Ene-2005 15:20:48              2005

SQL> select sysdate as Hoy, extract(month from sysdate) as Mes
2 from dual;

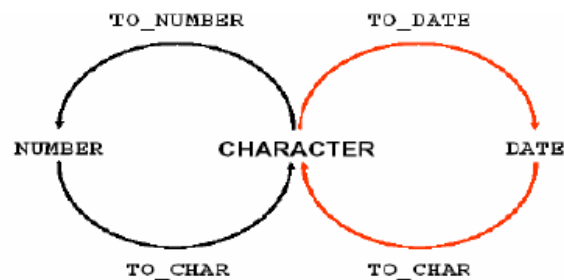
HOY                                MES
-----
26-Ene-2005 15:21:38              1

SQL> select sysdate as Hoy, extract(day from sysdate) as Día
2 from dual;

HOY                                DÍA
-----
26-Ene-2005 15:22:27              26
```



CANBION DE TIPOS DE DATOS



9	Representa un número
0	Fuerza a que se muestre un cero
\$	Coloca un signo flotante de dólar
L	Usa el símbolo flotante de moneda local
.	Imprime un punto decimal
,	Imprime un indicador de miles

```
SELECT last_name,  
       TO_CHAR(hire_date, 'fmDD Month YYYY')  
       AS HIREDATE  
FROM   employees;
```

LAST_NAME	HIREDATE
King	17 June 1987
Kochhar	21 September 1989
De Haan	13 January 1993
Hunold	3 January 1990
Ernst	21 May 1991
Lorentz	7 February 1999
Mourgos	16 November 1999

```
SELECT TO_CHAR(salary, '$99,999.00') SALARY  
FROM   employees  
WHERE  last_name = 'Ernst';
```

SALARY
\$6,000.00



FUNCIÓN NVL

Convierte un campo nulo al valor que se señale en el parámetro

```
SELECT last name, salary, NVL(commission_pct, 0)
       (salary*12) + (salary*12*NVL(commission_pct, 0)) AN SAL
FROM employees;
```

LAST_NAME	SALARY	NVL(COMMISSION_PCT,0)	AN_SAL
King	24000	0	288000
Kochhar	17000	0	204000
De Haan	17000	0	204000
Hunold	9000	0	108000
Ernst	6000	0	72000
Lorentz	4200	0	50400
Mourgos	5800	0	69600
Rajs	3500	0	42000

...
20 rows selected.

FUNCIÓN NVL2

Reemplaza un valor no nulo por un primer parámetro y si es nulo lo reemplaza por el otro parámetro

```
SELECT last name, salary, commission_pct
       NVL2(commission_pct,
            'SAL+COMM', 'SAL') income
FROM employees WHERE department_id IN (50, 80);
```

LAST_NAME	SALARY	COMMISSION_PCT	INCOME
Zlotkey	10500	.2	SAL+COMM
Abel	11000	.3	SAL+COMM
Taylor	8800	.2	SAL+COMM
Mourgos	5800		SAL
Rajs	3500		SAL
Davies	3100		SAL
Matos	2600		SAL
Vargas	2500		SAL

8 rows selected.



FUNCION NULLIF

Compara 2 expresiones, si son iguales retorna Null, de lo contrario retorna el valor de la primera expresión

1

```
SELECT first_name, LENGTH(first_name) "expr1",  
       last_name, LENGTH(last_name) "expr2",  
       NULLIF(LENGTH(first_name), LENGTH(last_name)) result  
FROM employees;
```

2

3

FIRST_NAME	expr1	LAST_NAME	expr2	RESULT
Steven	6	King	4	6
Neena	5	Kochhar	7	6
Lex	3	De Haan	7	3
Alexander	9	Hunold	6	9
Bruce	5	Ernst	5	
Diana	5	Lorentz	7	6
Kevin	5	Mourgos	7	6
Trenna	6	Rajs	4	6
Curtis	6	Davies	6	

...
20 rows selected.

1

2

3

FUNCIÓN COALESCE

Evalua un campo, si este no es NULL, devuelve el valor 1, si no, devuelve el valor 2

```
SELECT last_name,  
       COALESCE(manager_id, commission_pct, -1) comm  
FROM employees  
ORDER BY commission_pct;
```

LAST_NAME	COMM
Grant	149
Zlotkey	100
Taylor	149
Abel	149
King	-1
Kochhar	100
De Haan	100

FUNCIÓN DECODE

Permite consultas condicionales haciendo el trabajo de un CASE o IF-ELSE

```
SELECT last_name, job_id, salary,  
       DECODE(job_id, 'IT_PROG', 1.10*salary,  
                'ST_CLERK', 1.15*salary,  
                'SA_REP', 1.20*salary,  
                salary)  
       REVISED_SALARY  
FROM employees;
```

LAST_NAME	JOB_ID	SALARY	REVISED_SALARY
Lorentz	IT_PROG	4200	4620
Mourgos	ST_MAN	6800	6800
Rajs	ST_CLERK	3500	4025
Gietz	AC_ACCOUNT	8300	8300



FUNCIONES DE GRUPO

Las funciones de grupo operan sobre conjuntos de filas para dar un resultado por grupo

- AVG (promedio)
- COUNT (Cuenta la cantidad de filas)
- MAX (Máximo valor de columna)
- MIN (Mínimo valor de columna)
- STDDEV (Desviación estándar)
- SUM (Suma los valores de columna)
- VARIANCE (Varianza)

```
SELECT [column,] group_function(column),...  
FROM table  
[WHERE condition]  
[GROUP BY column_list]  
[HAVING group_function(column)]  
[ORDER BY column_list [ASC|DESC]];
```

```
SELECT AVG(salary), MAX(salary),  
       MIN(salary), SUM(salary)  
FROM   employees  
WHERE  job_id LIKE '%REP%';
```

AVG(SALARY)	MAX(SALARY)	MIN(SALARY)	SUM(SALARY)
8190	11000	6000	30600

COUNT

COUNT(*) cuenta la cantidad de filas de una tabla

```
SELECT COUNT(*)  
FROM   employees  
WHERE  department_id = 50;
```

COUNT(*)
5

COUNT(expr) cuenta la cantidad de filas con valores no nulos que cumplan con la expresión.

```
SELECT COUNT(commission_pct)  
FROM   employees  
WHERE  department_id = 80;
```

COUNT(COMMISSION_PCT)
3

COUNT(DISTINCT expr) cuenta el número de distintos valores no nulos de la expresión

```
SELECT COUNT(DISTINCT department_id)  
FROM   employees;
```

COUNT(DISTINCTDEPARTMENT_ID)
7



MANEJO DE VALORES NULOS EN LAS FUNCIONES DE GRUPO

Las funciones de grupo excluyen a los valores nulos

<pre>SELECT AVG(commission_pct) FROM employees;</pre>	
AVG(COMMISSION_PCT)	
	2125

La función NVL fuerza a las funciones de grupo a incluir los valores nulos

<pre>SELECT AVG(NVL(commission_pct, 0)) FROM employees;</pre>	
AVG(NVL(COMMISSION_PCT,0))	
	0425

AGRUPACIÓN DE DATOS

GROUP BY divide las filas en pequeños grupos

<pre>SELECT department_id, AVG(salary) FROM employees GROUP BY department_id ;</pre>	
DEPARTMENT_ID	AVG(SALARY)
10	4400
20	9600
50	3600
60	6400
80	10033.3333

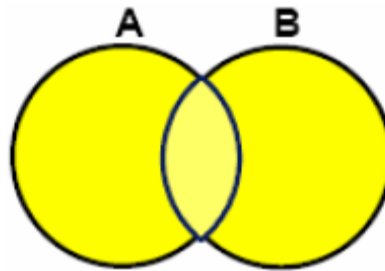
RESTRINGIR LOS RESULTADOS DE LOS GRUPOS

HAVING condiciona los resultados cuando se utilizan funciones de grupo

<pre>SELECT department_id, MAX(salary) FROM employees GROUP BY department_id HAVING MAX(salary) > 10000 ;</pre>	
DEPARTMENT_ID	MAX(SALARY)
20	13000
80	11000
90	24000
110	12000



OPERADOR UNION



Devuelve todos los resultados de las consultas involucradas pero eliminando los datos duplicados

```
SELECT employee_id, job_id
FROM employees
UNION
SELECT employee_id, job_id
FROM job_history;
```

EMPLOYEE_ID	JOB_ID
100	AD_PRES
101	AC_ACCOUNT
...	
200	AC_ACCOUNT
200	AD_ASST
...	
205	AC_MGR
206	AC_ACCOUNT

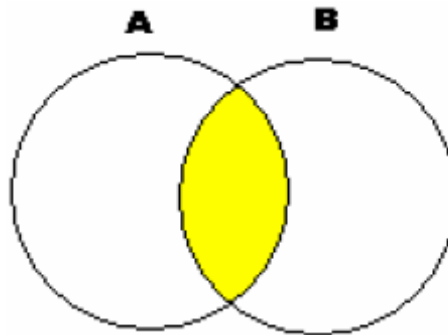
OPERADOR UNION ALL

Devuelve a todos los resultados de las consultas involucradas incluyendo a todas las duplicaciones.

```
SELECT employee_id, job_id, department_id
FROM employees
UNION ALL
SELECT employee_id, job_id, department_id
FROM job_history
ORDER BY employee_id;
```

EMPLOYEE_ID	JOB_ID	DEPARTMENT_ID
100	AD_PRES	90
101	AD_VP	90
...		
200	AD_ASST	10
200	AD_ASST	90
200	AC_ACCOUNT	90
...		
205	AC_MGR	110
206	AC_ACCOUNT	110

OPERADOR INTERSECT

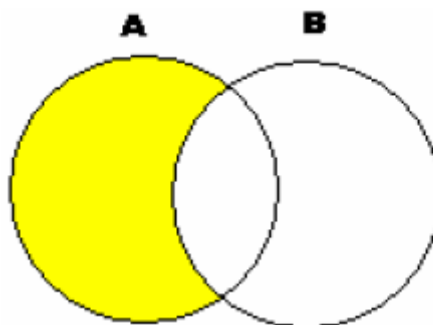


Este operador solo devuelve los resultados que son comunes a las consultas involucradas.

```
SELECT employee_id, job_id
FROM employees
INTERSECT
SELECT employee_id, job_id
FROM job_history;
```

EMPLOYEE_ID	JOB_ID
176	SA_REP
200	AD_ASST

OPERADOR MINUS



Solo devuelve los resultados que se encuentran en la primera consulta y que no se encuentren en la segunda consulta.

```
SELECT employee_id, job_id
FROM employees
MINUS
SELECT employee_id, job_id
FROM job_history;
```

EMPLOYEE_ID	JOB_ID
100	AD_PRES
101	AD_VP
102	AD_VP
103	IT_PROG
...	
201	MK_MAN
202	MK_REP
205	AC_MGR
206	AC_ACCOUNT



TOP – N

Las consultas Top-N piden por los n valores más grandes o más pequeños de una columna

```
SELECT [column_list], ROWNUM
FROM   (SELECT [column_list]
        FROM table
        ORDER BY Top-N_column)
WHERE  ROWNUM <= N;
```

1 2 3

```
SELECT ROWNUM as RANK, last_name, salary
FROM   (SELECT last_name, salary FROM employees
        ORDER BY salary DESC)
WHERE  ROWNUM <= 3;
```

RANK	LAST_NAME	SALARY
1	King	24000
2	Kochhar	17000
3	De Haan	17000



ROLLUP

Es una extensión de la cláusula GROUP BY para producir acumulativos, tales como los subtotales.

```
SELECT department_id, job_id, SUM(salary)
FROM   employees
WHERE  department_id < 60
GROUP BY ROLLUP(department_id, job_id);
```

DEPARTMENT ID	JOB ID	SUM(SALARY)
10	AD_ASST	4400
10		4400
20	MK_MAN	13000
20	MK_REP	6000
20		19000
30	PU_MAN	11000
30	PU_CLERK	13900
30		24900
40	HR_REP	6500
40		6500
50	ST_MAN	36400
50	SH_CLERK	64300
50	ST_CLERK	55700
50		156400
		211200

1

2

3

Referencias

<http://www-db.stanford.edu/~ullman/fcdb.html>